	`Symbol.iterator`, etc.).	calls; utilize `Reflect` API for meta-operations.	libraries/frameworks; design extensible APIs with `Symbol` and meta-programming.

With a grounding in functional programming principles and patterns, let's explore some common interview questions to see how these ideas are applied in interviews.

Question (fundamental skill): "What is a pure function, and how does `map` fit into functional programming?"

Analysis: This question evaluates understanding of pure functions, the ability to identify them in code, and knowledge of how methods like `map` reflect functional programming paradigms. Strong candidates will not only define pure functions but also articulate why immutability and side-effect-free transformations are critical for writing maintainable and predictable JavaScript. A thoughtful answer links these concepts to testability, debugging simplicity, and the referential transparency prized in functional programming.

Answer: A pure function is one that always produces the same output for the same input and does not cause any side effects. It does not modify external state or depend on variables outside its scope. This determinism makes pure functions predictable, easy to test, and ideal for functional programming. Listing 3.5 compares an impure function with a pure function.

Listing 3.5 Impure function vs. pure function

```
let counter = 0;
function increment() { #A
  counter++;
  return counter;
}
function add(x, y) { #B
  return x + y;
}
```

Listing 3.6 demonstrates how `map`, a builtin array method, embodies